

12th Progress Report

W. Chan

Date:	24/04/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- Went over new research objective and questions:
 1. Should remove the “online” word from the objective; online is more a conclusion from answering the first research question.
 2. Question 2a) is not very grounded; it suddenly states “... reinforcement learning algorithm studied.” without a prior question explicitly requiring a selection of one particular algorithm, would be better to find a way to ground this.
 3. Rewording in question 2a) from “... augmentations be made to ...” to “... augmentations be integrated to ...”.
 4. Rewording in question 3b) from “... and degree of fault tolerance...” to “... and fault tolerance...”; 3d) from “...nominal and fault flight performance...” to “... nominal performance and fault tolerance...”.

Also realised that maybe the research objectives/questions are not 100% representative of my intentions with my thesis, which is to come up with fault-tolerant controllers through improving current reinforcement learning algorithms, **whilst** evaluating the effects of said improvements.

- Finalized decision to proceed with Citation instead of the Flying V (💔 no. 1).
- Discussed the “Optimization problemification of the hyperparameter tuning problem”, and Erik’s input on how tuning could be done in stages in research like this: initial coarse phase to weed out the bad performing ideas, then a fine grain phase to optimize the performance of one/few algorithms.
- Went over the observation that for the cg-shift fault case, non-zero critic learning rate results in divergence. Realised implementation of cg shift was wrong (did not account for changes to control effectiveness!).
- Went over the hyper-parameter tuning results, which gave (as definitive as can be obtained) conclusions on which algorithm augmentations improve performance and which don’t. Note that the results were from the elevator reversal fault case, slightly different findings might be found for the nominal flight case. The observations are namely:
 1. IDHP with replacing trace (IDHP-RT) trivially deteriorates performance.
 2. IDHP with accumulating trace (IDHP-AT) improves performance.

3. Multistep IDHP (MIDHP) deteriorates performance (💔 no. 2).
4. MIDHP-RT improves performance (😞 !?), it in fact has best performance.
5. MIDHP-AT deteriorates performance.

Looking at the scatter plots, it would be interesting to do some data clustering to see if there exists relationships between hyperparameters and metrics (very exciting but not sure how to do this/how to get useful results from this yet).

Description of the results since last meeting

- **Further investigated shift cg fault case**

Ran the agents on two cases: shifting the cg from the start, and shifting the cg at 20 seconds. The agent used the same hyperparameters and RNG seeds in both cases, with the low critic learning rate being non-zero (specifically 0.01), results are plotted in Figure 1.

It was found that when the cg is shifted from the start, the agent has no problem learning a stable policy/controller, whereas shifting cg midflight causes divergence/instability issues.

- **Changes to Multi-step:**

Suddenly remembered that in the last meeting, I forgot to report the observation that critic weights are consistently more likely to diverge than actor weights. Consequently, when eligibility traces were applied to the critic, the rate at which critic weights diverged increased, the hypothesis behind this is that since the weight updates critics were receiving were already borderline divergent, using traces to accumulate these updates made the final updates even more divergent. Thus a decision was made, and the results presented last week only had eligibility traces applied to the actor.

Remembering this led me to question whether it would be wiser to apply multi-step to only the actor or critic, as opposed to both. Furthermore, in the dynamic programming context, the original idea of multi-step updates came from performing a means of approximating the policy iteration algorithm by performing **policy evaluation** multiple times; in the TD algorithm context, the idea of multistep is to bootstrap value function estimates using multiple time steps' worth of rewards [1, 2]. This hints that the multi-step idea in the IDHP case might serve better when applied only to performing critic updates, which is analogous to the policy evaluation step or TD bootstrapping.

Making this small change, and then running the 200+ hyperparameter configurations from last week's report for this updated MIDHP (referred to for now as M(c)IDHP), we can obtain the updated metric scatter plot shown in Figure 2. From this plot, it can be seen that in the reversed elevator scenario, using multi-step updates for the critic, in general, is more performant than vanilla IDHP or full MIDHP.

- **Ran the best hyperparameters found from Figure 2 for the shifted cg and damped elevator scenarios.**

For the non-multistep algorithms in the cg shift/damped elevator cases, the best hyperparameters for the reversed elevator case also had good convergence properties but not the best tracking performance. The observation here is that in general, the hyperparameters with the best tracking performance in the shifted cg and damped elevator cases, do not have good tracking performances or even convergence properties in the reversed elevator case. To give an example, if we run MIDHP with the hyperparameters tuned on the shift cg scenario on both the shift cg and reversed elevator scenarios we see the results shown in Figure 3. Here, the controller does not adapt to a reversed elevator well but adapts to a shift in the cg very well as evidenced by the increased elevator deflections.

Going further in this observation, the opposite case is also shown in Figure 4, where the hyperparameters optimal for the reversed elevator fault are run on the cg shift fault: Optimal hyperparameter in the reversed elevator is not optimal in other cases.

- **Picking the representative hyperparameters**

The first rule of thumb used to pick the hyperparameter which will represent each algorithm, is to pick the one which performed best in the reversed elevator scenario for each algorithm. For example, The hyperparameters shown in the light green box in the IDHP plot in Figure 2 yielded the best set of runs for IDHP, the performance of this hyperparameter in the other 3 flight cases (nominal, shift cg, damped elevator) is checked to see if they reasonably converged, if so then that hyperparameter is selected to represent the algorithm.

In other algorithms, however, this became a little more tricky. For IDHP-AT, the best hyperparameters in the reversed elevator case are shown in the dark green box in Figure 2, but checking the performance of this hyperparameter in the shift cg case reveals that it yields occasionally diverging controllers, as shown in Figure 5. This is observed for all the multistep augmented algorithms; so for these algorithms, the choice has to be made on whether to pick the hyperparameters which perform best when the elevator is reversed, or when cg is shifted/elevator is damped. As of now, the decision is to use the best reversed elevator hyperparameters for MIDHP-RT (since that one had 0 diverged runs on reversed elevator!), and for MIDHP and MIDHP-AT *try to* use hyperparameters that perform better on shift cg/damped elevator.

- **Worked on preliminary thesis**

Working on writing the MDP environment specification in the preliminary report, describing the short period model, MDP state and action space, fault cases, ... etc.

Spent approximately 2 hours making a figure and learning how to embed tex script in pdfs, the figure looks (in my opinion) pretty cool though.

- **Iterated on the research objective and question**

1. Removed the word “online” in the research objective.
2. Reworded question 1 entirely (not the subquestions), the original question was: “What reinforcement learning algorithm can yield a flight controller which is the most tolerant to faults?” I feel the new question gives more grounding to question 2, and also the question sounds a bit more natural now.
3. Added subquestion 3b.

Research Objective

To advance the state-of-the-art reinforcement learning based flight controllers and improve their fault tolerance, by researching novel augmentations to reinforcement learning algorithms.

Research Questions

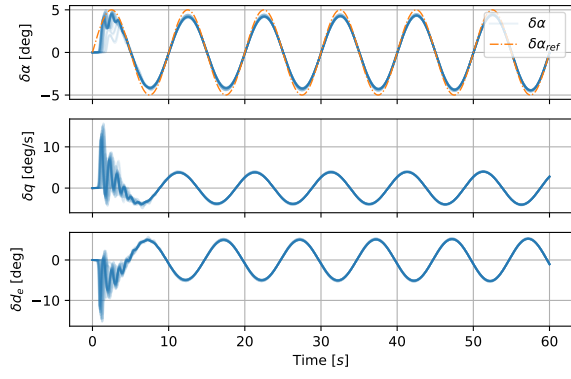
1. What promising reinforcement learning algorithm for fault tolerance and tracking performance should be further studied in the present research?
 - (a) What RL algorithms are considered to be state-of-the-art?
 - (b) How is fault tolerance defined and tested in past research?
 - (c) Which algorithms have been shown to provide the best fault tolerance?
 - (d) What reference tracking performance have these algorithms shown in past research?
 - (e) What promising augmentations to reinforcement learning algorithms can be made and experimented with?
2. How can the identified algorithm be applied to control the PH-LAB research aircraft?
 - (a) How can the identified augmentations be made to the reinforcement learning algorithm studied?
 - (b) How should the flight control system be structured?
 - (c) What are the variables defining the MDP in the case of controlling the PH-LAB?
3. How does the developed flight controller perform during nominal flight and in the presence of faults?
 - (a) What flight scenarios should be designed to test the proposed controller's nominal performance and fault tolerance?
 - (b) How should a controller's nominal performance and fault tolerance be measured?
 - (c) What is the implemented controller's nominal performance and fault tolerance?
 - (d) How do the proposed augmentations affect the characteristics of the baseline reinforcement learning algorithm?
 - (e) How well does the proposed controller's nominal performance and fault tolerance compare to a traditional PID controller?

- **Regarding mismatch between thesis intention and objectives:**

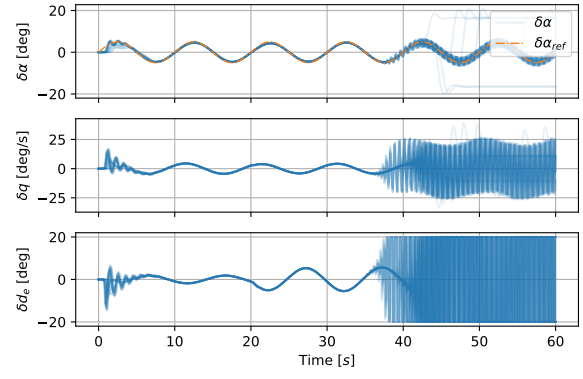
I would like to try and showcase the effect of augmentations on IDHP while not making that the sole outcome of the thesis, i.e. I want my overall contribution to be the combination of the final controller and the comparison of augmented vs vanilla IDHP. And this, I think, is represented by the current research questions (3c, 3d, and very loosely 3e).

Plans for next 2 weeks

1. Did not have a chance to do data/hyperparameter clustering yet, but the old idea is to find clusters of samples in hyperparameter space and then colour these clusters in metric space to see how different hyperparameter sets map to performance metrics. Am curious to do this but will only do it if time allows.
2. Continue with writing the preliminary thesis, working on the IDHP theory sections soon.
3. Pick the representative hyperparameters for MIDHP-X algorithms.

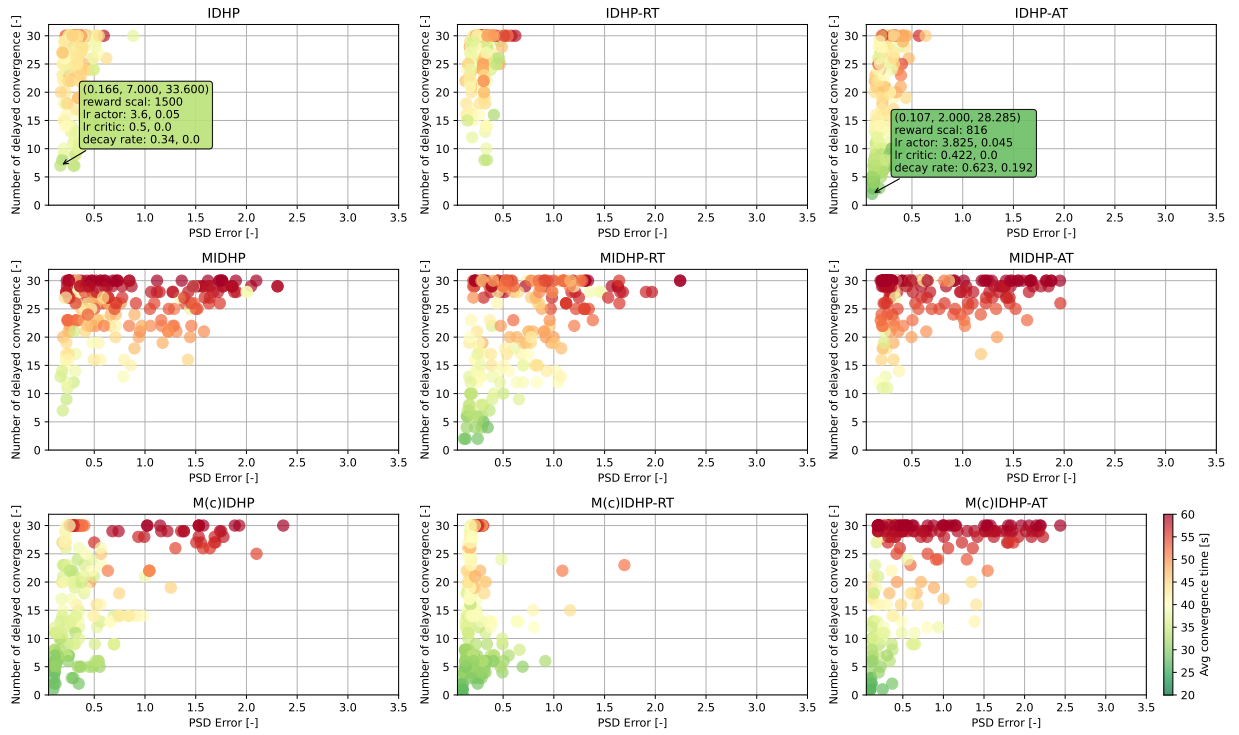


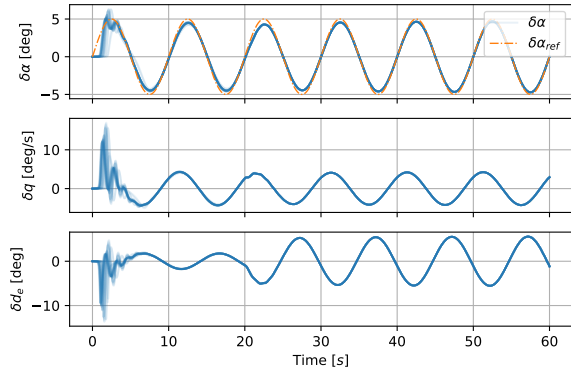
(a) Shift cg at 0s.



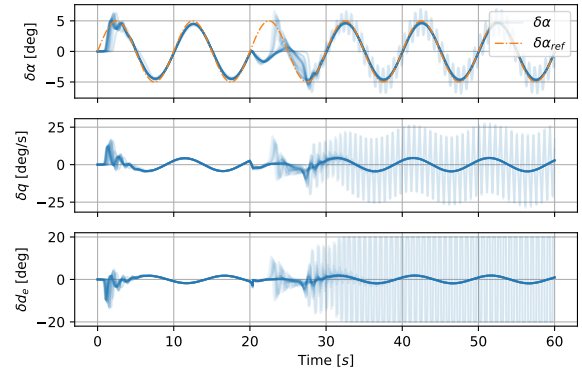
(b) Shift cg at 20s.

Figure 1: State action time trace of IDHP in the cg shift test.

Figure 2: Metric scatter plot of all algorithms, first row is IDHP with **no multistep**, second row is IDHP with multistep applied to **actor and critic**, third row is IDHP with multistep applied to **only critic**.

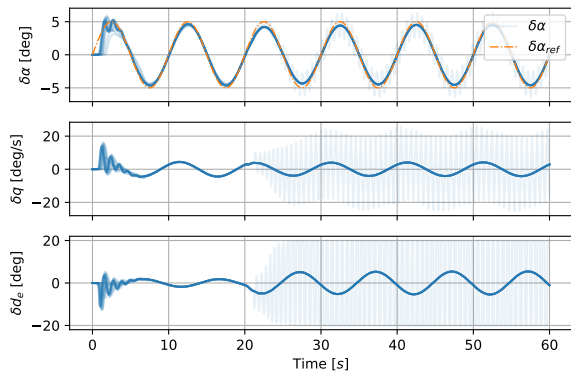


(a) Shift cg at 20s.

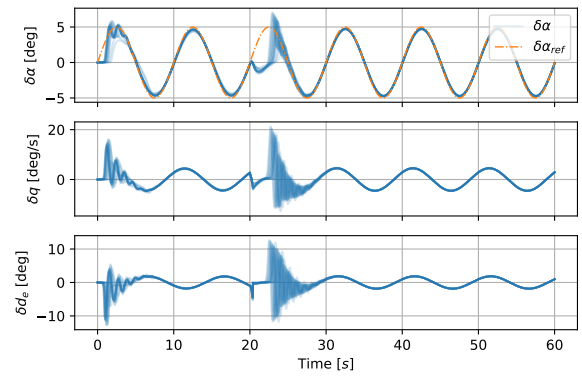


(b) Reversed elevator at 20s.

Figure 3: State action time trace in two flight scenarios of the MIDHP controller using hyperparameters optimized for the **cg shift** scenario.



(a) Shift cg at 20s.



(b) Reversed elevator at 20s.

Figure 4: State action time trace in two flight scenarios of the MIDHP-RT controller using hyperparameters optimized for the **reversed elevator** scenario.

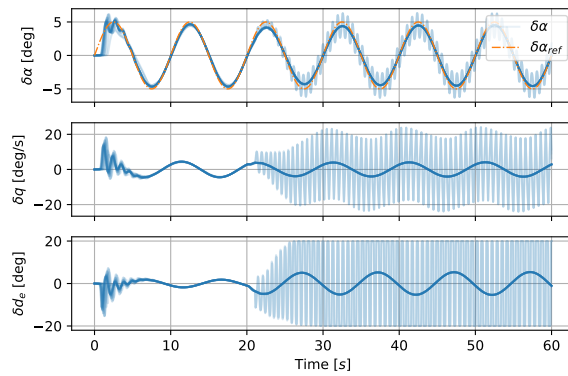


Figure 5: IDHP-AT using most optimal hyperparameters from the reversed elevator case on the shift cg case, cg shifts at 20s.

References

- [1] R. Sutton and A. Barto. *Reinforcement Learning, An Introduction*. 2nd ed. Cambridge, Massachusetts: The MIT Press, 2018.

- [2] Biao Luo, Derong Liu, Tingwen Huang, Xiong Yang, and Hongwen Ma. “Multi-step heuristic dynamic programming for optimal control of nonlinear discrete-time systems”. In: *Information Sciences* 411 (2017), pp. 66–83. ISSN: 0020-0255. DOI: <https://doi.org/10.1016/j.ins.2017.05.005>.